WORD-LEVEL PLAGIARISM CHECKER

DSA-3 Project Proposal

25CS2103E — Data Structures and Algorithms (DSA-3)
2nd Year | Department: AIDS
Team: D Tanmai and V Akshitha | Guide: Chandusha

Problem Statement

Why a lightweight, explainable plagiarism checker?

The problem

Copy-paste detection alone misses content that is lightly reworded, reordered, or paraphrased.

The gap

Commercial tools can be costly, cloud-dependent, and opaque about how similarity is calculated.

Our solution

Build a first-principles checker for short documents using hashing + dynamic programming.

Project Overview

Detects both verbatim copying and paraphrased/reordered overlap in short documents.
Uses Rabin–Karp rolling hashes for fast exact-match detection.
Uses Longest Common Subsequence (LCS) dynamic programming to find non-contiguous similarity.
Reports an overall similarity score and highlights matched passages.
Attributes each flagged span to its most likely reference source.
All core matching and scoring steps are implemented from first principles.

Objectives

01 Preprocess

Tokenise, normalise and clean documents.

02 Exact Match

Implement Rabin–Karp rolling-hash detection.

03 Paraphrase

Use LCS / edit-distance based dynamic programming.

04 Score

Calculate document-pair similarity and source attribution.

05 Report

Highlight matched spans in an easy-to-read report.

06 Evaluate

Measure precision, recall and false-positive rate.

System Architecture

Five-stage pipeline



Two-stage strategy: cheap hashing first → expensive DP only for shortlisted candidates

Methodology — Core DSA Concepts

1. Text Preprocessing

Split documents into words, convert to lowercase, remove punctuation, optionally remove stop words, and retain original positions.

2. Rabin–Karp Rolling Hash

Hash k-word windows from the corpus. Look up submission hashes in a hash table and verify matches token-by-token.

3. LCS Dynamic Programming

For shortlisted pairs, compare token sequences to find long common subsequences that exact hashing can miss.

4. Scoring & Reporting

Combine exact and LCS matches, calculate similarity, identify the strongest source, and highlight matched spans.

Algorithm 1 — Rabin–Karp

Fast detection of verbatim copied passages

Choose a window of k consecutive words.
Compute a polynomial rolling hash for every window in the reference corpus.
Store hash → document ID + position in a hash table.
Slide the same window through the submitted document and update the hash efficiently.
On a hash match, compare the actual tokens to eliminate collision errors.
Average lookup is O(1) per window; overall average matching is approximately O(n + m) for a pattern/document comparison.

Example

Reference:
“data structures improve program efficiency”

Submission:
“data structures improve program efficiency greatly”

A matching k-word window produces the same verified token sequence.

Algorithm 2 — LCS Dynamic Programming

Finding similarity that exact matching misses

LCS finds the longest sequence of tokens appearing in both documents in the same relative order.
If tokens match: DP[i][j] = DP[i−1][j−1] + 1.
If they do not match: DP[i][j] = max(DP[i−1][j], DP[i][j−1]).
Backtracking the DP table reveals aligned common spans.
A similarity threshold is used to decide whether a span is significant enough to flag.

0	0	0	0	0
0	1	2	3	0
0	2	0	2	0
0	3	2	1	0

Illustrative LCS DP table

Scoring & Reporting

Similarity Score

Combine tokens covered by verified exact matches and significant LCS-aligned spans.

Example:
Matched tokens / total submission tokens × 100

Source Attribution

For every flagged span, select the reference document with the strongest verified match and record its position.

Visual Report

Show overall percentage plus highlighted passages, source document, and matched span details.

Evaluation Plan

Test Set

Create a small hand-labelled corpus containing:
- Original documents
- Verbatim copies
- Paraphrased documents
- Mixed / partially copied documents

Metrics

Precision = correct plagiarism flags / all flags

Recall = detected plagiarism / actual plagiarism

Also track false-positive rate.

Analysis

Study under-detection and over-detection cases, then discuss threshold tuning and possible future improvements.

Literature Survey

Key ideas that shaped the design

Study / Source	Key Contribution	Limitation
Karp & Rabin (1987)	Rabin–Karp rolling hash	Fast exact matching; collisions need verification
Manber & Myers (1993)	Suffix arrays	Powerful substring search; high implementation overhead
MOSS (Schleimer et al., 2003)	Winnowing / fingerprints	Efficient shortlist strategy; designed mainly for source code
Hunt & McIlroy (1976)	LCS / diff	Strong sequence comparison; quadratic worst-case cost
Turnitin / Grammarly	Real-world plagiarism systems	Good reporting expectations; closed-source/cloud dependent

Software & Hardware Requirements

Software

OS: Windows 10/11, Ubuntu 20.04+, or macOS
Language: Java / Python / C++
IDE: VS Code / Eclipse / PyCharm / IntelliJ
Version control: Git + GitHub/GitLab
Libraries: standard I/O, file handling, basic strings only
Testing: JUnit / PyTest
Docs/diagrams: MS Word / LaTeX, draw.io / Lucidchart

Hardware

Processor: Dual-core 2.0 GHz+
RAM: 4 GB minimum; 8 GB recommended
Storage: 500 MB free
Display: 1366×768+
Internet: only for setup, documentation and version control

Team & 10-Week Timeline

Suggested plan for a 4-member team



Weeks 1–2 Requirements, dataset collection, preprocessing

Weeks 3–4 Rabin–Karp exact matching + unit tests

Weeks 5–6 LCS-based paraphrase detection

Weeks 7–8 Scoring, source attribution, report generation

Weeks 9–10 Integration, evaluation, documentation & demo

DSA Concepts Demonstrated

Hashing

Rabin–Karp rolling polynomial hash + hash table

String Processing

Tokenisation, normalisation and word-level windows

Dynamic Programming

LCS table construction and backtracking

Complexity

Trade-off between fast shortlist and expensive verification

Data Structures

Arrays/lists, hash tables, DP matrices

Algorithm Design

Two-stage filtering + verification strategy

Expected Outcome

A working short-document plagiarism checker built from first principles.
Fast exact-match detection through Rabin–Karp.
More flexible similarity detection through LCS dynamic programming.
Transparent similarity scores, highlighted matches, and source attribution.
A practical demonstration of core DSA concepts in a real-world application.

Thank You | Questions & Discussion